

Informe Técnico

Servicio MLOps end-to-end: Predicción del Nivel de Obesidad

Curso	MLOPS Sec.2 FIC - Educación Ejecutiva FIC-Ejecutiva 2026
Docente	Ahmad Armoush Armoush
Integrantes	Andrés López , Cristóbal Barrientos, Cristóbal Crespo.
Repositorio	github.com/turbogarypleto/prediccion_obesidad_mlops
Tag de entrega	v1.0.0
Fecha	18 de agosto de 2026

1. Problema, datos y modelo

El problema es de clasificación multiclase: a partir de 16 variables de una persona (género, edad, altura, peso, antecedentes familiares de obesidad, hábitos alimenticios, actividad física y medio de transporte habitual) se predice su categoría de nivel de obesidad (Normal_Weight, Overweight_Level_I/II, Obesity_Type_I/II/III, entre otras).

Los datos son 2.111 registros sin información personal identificable, correspondientes al dataset "Estimation of obesity levels based on eating habits and physical condition" del UCI Machine Learning Repository (Palechory y De la Hoz Manotas, 2019, DOI: 10.24432/C5H31Z), también disponible como réplica en Kaggle. El 77% de los registros es sintético, generado con la técnica SMOTE, y el 23% corresponde a datos reales recolectados mediante una plataforma web de personas de México, Perú y Colombia. El conjunto se dividió en 80% entrenamiento (1.688 registros) y 20% prueba (423 registros), de forma estratificada por la variable objetivo y con semilla fija (random_state=42) para que el resultado sea reproducible.

El modelo es un RandomForestClassifier de scikit-learn con 100 árboles. Las 8 variables categóricas se codifican con LabelEncoder, uno por columna, y se guardan junto al modelo para que la codificación usada en producción sea exactamente la misma que se usó al entrenar. El entrenamiento corre desde un script (training/train.py), no desde un notebook, y deja tres artefactos versionados en models/: el modelo (model.pkl), los codificadores (encoders.pkl) y una metadata (metadata.json) con las features usadas, la fecha de entrenamiento y el accuracy sobre datos no vistos.

2. Decisiones de diseño

El servicio expone cinco endpoints con FastAPI: GET / (información general), GET /health (estado del servicio y del modelo cargado), GET /model/schema (metadata del modelo), POST /predict y POST /predict/batch. El modelo, los encoders y la metadata se cargan una sola vez al iniciar la aplicación, no en cada solicitud.

Las entradas se validan con modelos Pydantic. Un campo faltante o de tipo incorrecto devuelve 422; un valor categórico fuera del vocabulario conocido devuelve 400 con la lista de valores válidos en el mensaje, para que quien consuma la API pueda corregir la solicitud sin mirar el código. Los campos numéricos (edad, altura, peso, escalas de hábitos) tienen además restricciones de rango (por ejemplo, edad entre 0 y 120) para rechazar valores absurdos antes de que lleguen al modelo.

La aplicación corre en una imagen Docker basada en python:3.10-slim, con dependencias fijadas por versión, un usuario no root y un HEALTHCHECK contra /health. El .dockerignore deja fuera del build todo lo que no se necesita en producción: tests, código de entrenamiento y el dataset crudo.

El pipeline de GitHub Actions (ci.yml) tiene cinco jobs encadenados con needs: lint (Ruff), test (pytest), build (construye la imagen y la guarda como artefacto), smoke (levanta un contenedor real desde ese artefacto y prueba /health y /predict con un payload de ejemplo) y deploy, que solo corre si los cuatro anteriores pasaron y dispara el despliegue en Render mediante un webhook guardado en GitHub Secrets. El servicio queda disponible en prediccion-obesidad-mlops.onrender.com.

3. Resultados

El modelo alcanzó un accuracy de 0,9527 sobre las 423 muestras de prueba, medido con accuracy_score de scikit-learn. El equipo escribió 9 pruebas con pytest que cubren el contrato de los endpoints, una predicción válida, un campo obligatorio faltante, un valor categórico fuera de rango, una predicción por lotes válida e inválida, y un tipo de dato incorrecto. Todas corren sin red ni credenciales, usando el TestClient de FastAPI en memoria.

El servicio quedó desplegado públicamente en Render (plan gratuito). GET /health responde 200 desde internet sobre HTTPS y POST /predict devuelve una predicción real con el mismo ejemplo documentado en el README. Al ser un plan gratuito, la instancia se duerme tras un rato sin tráfico y la primera consulta después de eso puede demorar hasta 50 segundos; las siguientes son rápidas.

4. Limitaciones y trabajo futuro

El modelo no tiene monitoreo de drift ni reentrenamiento automático: si la distribución de los datos de entrada cambia con el tiempo, nadie lo detecta hoy. Tampoco hay autenticación en los endpoints, así que cualquiera con la URL puede consultar el servicio, y no se registran las predicciones servidas, lo que dificulta auditar el comportamiento del modelo más adelante.

Como trabajo futuro queda pendiente un registro de modelos con MLflow en vez de empaquetar el artefacto dentro de la imagen, exponer métricas del servicio (latencia, conteo de requests, tasa de error), y agregar detección de drift comparando la distribución de las solicitudes contra la del set de entrenamiento.

5. Distribución del trabajo

- **Andrés López:** definió el problema y el dataset, armó la estructura del repositorio (carpetas, API, Docker), lo publicó en GitHub y escribió la primera versión principal del README.

- **Cristóbal Barrientos:** entrenó el modelo de clasificación (Random Forest), ajustando sus hiperparámetros y validando el desempeño sobre el conjunto de prueba hasta alcanzar el accuracy reportado en este informe.
- **Cristóbal Crespo:** construyó el pipeline de CI/CD (lint, test, build, smoke y despliegue encadenados), agregó validación de rangos en las entradas de la API, versionó la metadata del modelo con fecha de entrenamiento, configuró y documentó el despliegue público automatizado en Render, y redactó el presente informe.